

EECE527
INTRODUCTION TO HAPTICS
LECTURE 14: COLLISION DETECTION (5)

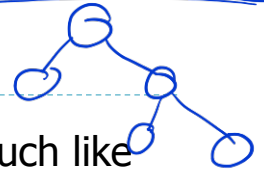


2011-04-26
Seungmoon Choi, Ph.D.

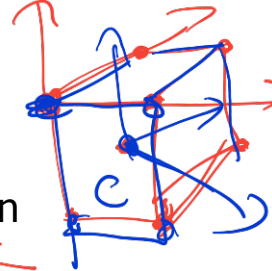
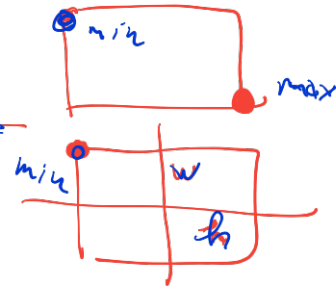
BOUNDING VOLUMES (2)

Oriented Bounding Boxes (OBBs)

- ▶ Oriented bounding box (OBB): A rectangular block, much like an AABB but with an arbitrary orientation.
- ▶ Representations for an OBB
 - ▶ A collection of eight vertices
 - ▶ A collection of six planes
 - ▶ A collection of three slabs (a pair of parallel planes)
 - ▶ A corner vertex plus three mutually orthogonal edge vectors
 - ▶ A center point plus an orientation matrix and three half-edge lengths.
- ▶ The last one is commonly the preferred representation for OBBs, as it allows for a much cheaper OBB-OBB intersection test than do the other representations.

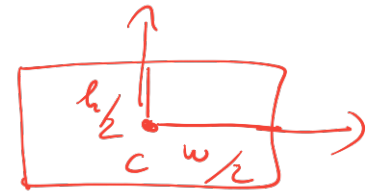


AABB



▶ 3

Representation of OBB



```
// Region R = { x | x = c+r*u[0]+s*u[1]+t*u[2] }, |r|<=e[0], |s|<=e[1], |t|<=e[2]
struct OBB {
    Point c;           // OBB center point
    Vector u[3];       // Local x-, y-, and z-axes
    Vector e;          // Positive halfwidth extents of OBB along each axis
};
```

- ▶ This representation requires 15 floats, or 60 bytes for IEEE single-precision floats.
- ▶ Thus, the OBB is quite an expensive bounding volume in terms of memory usage.
- ▶ The memory requirements could be lowered by storing the orientation not as a matrix but as Euler angles or as a quaternion, using three to four floating-point components instead of nine.
- ▶ Unfortunately, for an OBB-OBB intersection test these representations must be converted back to a matrix for use in the effective separating axis test, which is a very expensive operation.

9 floats

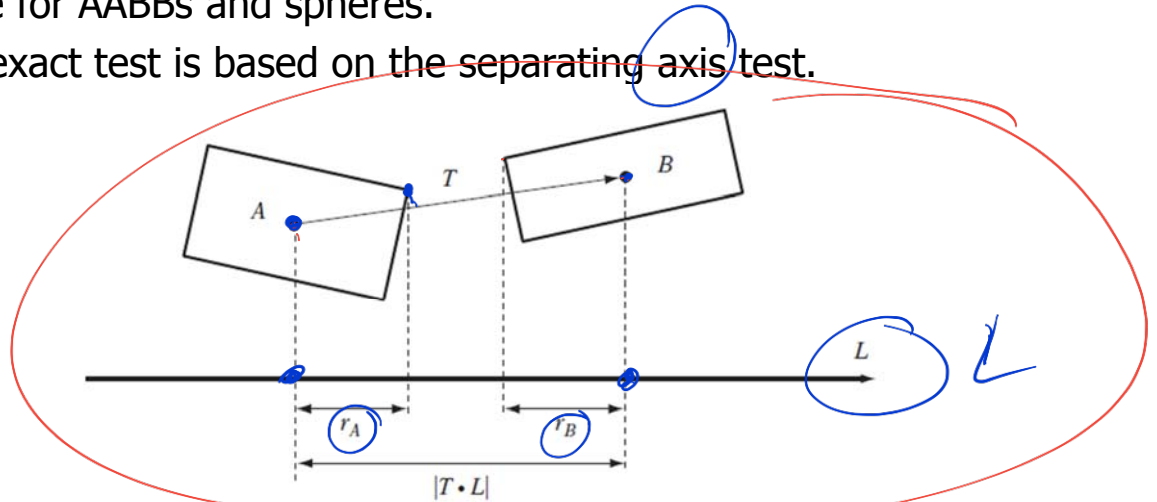
$\frac{C}{R} \frac{3 \times 1}{1}$

4 floats

▶ 4

OBB-OBB Intersection (1)

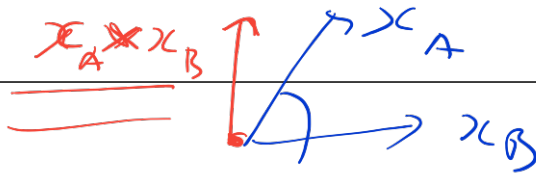
- ▶ OBB-OBB intersection tests are much more complicated than those for AABBs and spheres.
- ▶ The exact test is based on the separating axis test.



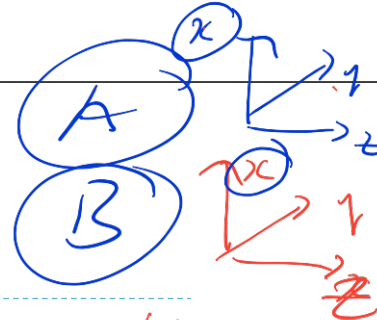
- ▶ The two objects A and B are separated if for some axis L

$$|T \cdot L| > r_A + r_B.$$

▶ 5

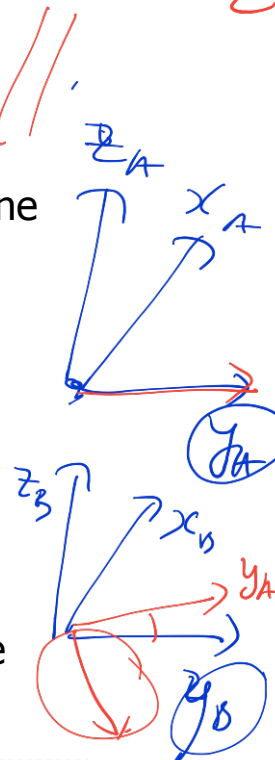


OBB
OBB



OBB-OBB Intersection (2)

- ▶ It can be shown that at most 15 separating axes must be tested to correctly determine the OBB overlap status.
- ▶ These axes are the three coordinate axes of A, the three coordinate axes of B, and the nine axes perpendicular to one coordinate axis of A and one coordinate axis of B.
- ▶ The number of operations in the test can be reduced by expressing B in the coordinate frame of A.
- ▶ For actual codes and their robustness issue, see the text.
- ▶ To make the OBB-OBB test as efficient as possible, it is important that the axes are tested in the order given in the next table.



▶ 6

OBB-OBB Intersection (3)

Table 4.1 The 15 separating axis tests needed to determine OBB-OBB intersection. Superscripts indicate which OBB the value comes from.

L	$ T \cdot L $	r_A	r_B
u_0^A	$ t_0 $	e_0^A	$e_0^B r_{00} + e_1^B r_{01} + e_2^B r_{02} $
u_1^A	$ t_1 $	e_1^A	$e_0^B r_{10} + e_1^B r_{11} + e_2^B r_{12} $
u_2^A	$ t_2 $	e_2^A	$e_0^B r_{20} + e_1^B r_{21} + e_2^B r_{22} $
u_0^B	$ t_0 r_{00} + t_1 r_{10} + t_2 r_{20} $	$e_0^A r_{00} + e_1^A r_{10} + e_2^A r_{20} $	e_0^B
u_1^B	$ t_0 r_{01} + t_1 r_{11} + t_2 r_{21} $	$e_0^A r_{01} + e_1^A r_{11} + e_2^A r_{21} $	e_1^B
u_2^B	$ t_0 r_{02} + t_1 r_{12} + t_2 r_{22} $	$e_0^A r_{02} + e_1^A r_{12} + e_2^A r_{22} $	e_2^B
$u_0^A \times u_0^B$	$ t_2 r_{10} - t_1 r_{20} $	$e_1^A r_{20} + e_2^A r_{10} $	$e_1^B r_{02} + e_2^B r_{01} $
$u_0^A \times u_1^B$	$ t_2 r_{11} - t_1 r_{21} $	$e_1^A r_{21} + e_2^A r_{11} $	$e_0^B r_{02} + e_2^B r_{00} $
$u_0^A \times u_2^B$	$ t_2 r_{12} - t_1 r_{22} $	$e_1^A r_{22} + e_2^A r_{12} $	$e_0^B r_{01} + e_1^B r_{00} $
$u_1^A \times u_0^B$	$ t_0 r_{20} - t_2 r_{00} $	$e_0^A r_{20} + e_2^A r_{00} $	$e_1^B r_{12} + e_2^B r_{11} $
$u_1^A \times u_1^B$	$ t_0 r_{21} - t_2 r_{01} $	$e_0^A r_{21} + e_2^A r_{01} $	$e_0^B r_{12} + e_2^B r_{10} $
$u_1^A \times u_2^B$	$ t_0 r_{22} - t_2 r_{02} $	$e_0^A r_{22} + e_2^A r_{02} $	$e_0^B r_{11} + e_1^B r_{10} $
$u_2^A \times u_0^B$	$ t_1 r_{00} - t_0 r_{10} $	$e_0^A r_{10} + e_1^A r_{00} $	$e_1^B r_{22} + e_2^B r_{21} $
$u_2^A \times u_1^B$	$ t_1 r_{01} - t_0 r_{11} $	$e_0^A r_{11} + e_1^A r_{01} $	$e_0^B r_{22} + e_2^B r_{20} $
$u_2^A \times u_2^B$	$ t_1 r_{02} - t_0 r_{12} $	$e_0^A r_{12} + e_1^A r_{02} $	$e_0^B r_{21} + e_1^B r_{20} $

r_{ij} : (i, j) element of rotation matrix R

dot product
 $A \cdot B = 0.6$

true/false

given

given

Computing a Tight OBB (1)

- Computing tight-fitting, optimal oriented bounding boxes is a difficult problem, made worse by the fact that the volume difference between a poorly aligned and a well-aligned OBB can be quite large.

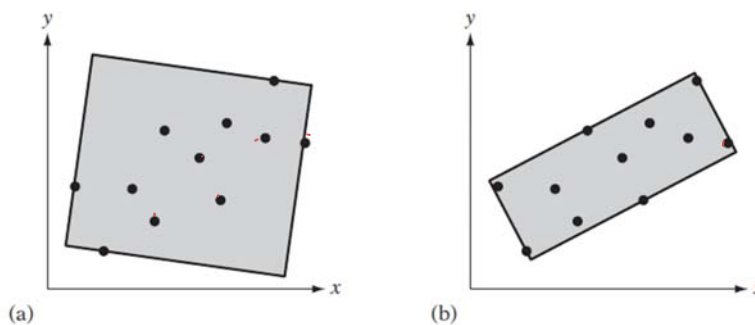
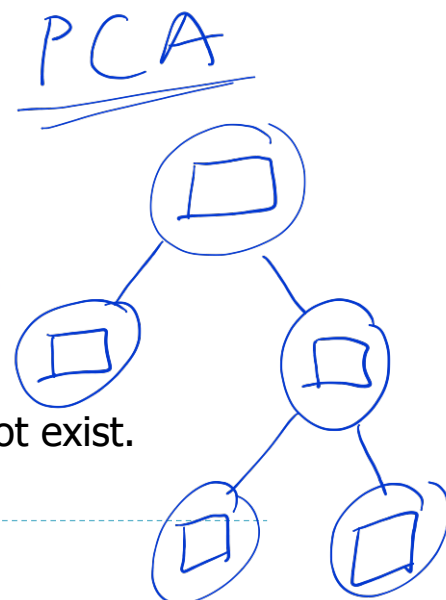


Figure 4.10 (a) A poorly aligned and (b) a well-aligned OBB.

- Theoretically exact and practical algorithms do not exist.



Computing a Tight OBB (2)

- ▶ For approximate algorithms, we can rely on PCA (Principal Component Analysis) to find the longest axes of an OBB.
- ▶ However, the covariance matrix is heavily dependent on the internal point distributions, which can lead to a bad selection of the principal axis.
- ▶ We can use only the points included in a convex hull to compute the covariance matrix, but vertex distributions on the convex hull can still be clustered.
- ▶ A good solution is to use a continuous formulation of covariance, computing the covariance across the entire face of the primitives [Gottschalk00; RAPID].
- ▶ If the convex hull is already available, this algorithm is $O(n)$.
- ▶ If the convex hull must be computed, it is $O(n \log n)$.

▶ 9

PCA - sphere

Computing a Tight OBB (3)

- ▶ Given n triangles (p_k, q_k, r_k) , $0 \leq k \leq n$, in the convex hull, the covariance matrix is given by

$$C_{ij} = \left(\frac{1}{a_H} \sum_{0 \leq k < n} \frac{a_k}{12} (9m_{k,i}m_{k,j} + p_{k,i}p_{k,j} + q_{k,i}q_{k,j} + r_{k,i}r_{k,j}) \right) - m_{H,i}m_{H,j}$$

where $a_k = \|(q_k - p_k) \times (r_k - p_k)\| / 2$ is the area and $m_k = (p_k + q_k + r_k) / 3$ is the centroid of triangle k .

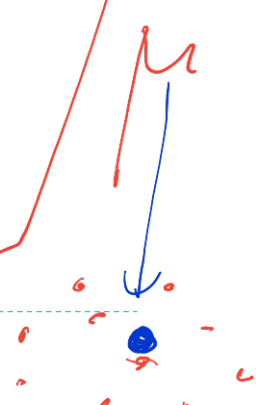
- ▶ The total area of the convex hull is given by

$$a_H = \sum_{0 \leq k < n} a_k$$

- ▶ The centroid of the convex hull is

$$m_H = \frac{1}{a_H} \sum_{0 \leq k < n} a_k m_k$$

▶ 10



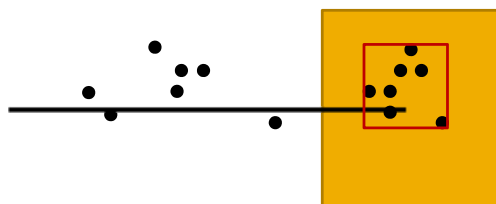
Computing a Tight OBB (3)

- ▶ From the covariance matrix, we can find the longest and shortest axes by computing its eigenvalues and eigenvectors.
- ▶ The last axis can be found to be perpendicular to the two axes.
- ▶ From these, we can find an OBB easily.
- ▶ In [Mirtich96a], a related method treating the polyhedron as a solid body is described.
- ▶ Assuming a uniform density polyhedron, Mirtich's polyhedral mass property algorithm integrates over the volume of the polyhedron, computing its 3×3 inertia tensor (also known as the inertia or mass matrix).
- ▶ The eigenvectors of this symmetric matrix are called the principal axes of inertia, and the eigenvalues the principal moments of inertia.

▶ 11

Computing a Tight OBB (4)

- ▶ To further optimize PCA-based OBBs, we can perform iterative optimization.
- ▶ Initial OBBs
 - ▶ All-principal-components box: All principle axes are found by PCA.
 - ▶ Max-principal-component box: The longest axis is found by PCA, and the other two axes are determined by the minimum bounding rectangle of the points projected from the vertices to the plane orthogonal to the longest axis.
 - ▶ Min-principal-component box: The shortest axis is found by PCA, and the rest of the same with the max-principal-component box.



▶ 12

Computing a Tight OBB (5)

- ▶ Iteration
 - ▶ Project all vertices to the plane that is orthogonal to one of the axes of the OBB.
 - ▶ Find the minimum bounding rectangle of the projected points.
 - ▶ Update the OBB by taking the two axes from the minimum bounding rectangle.
 - ▶ Repeat the above if the volume of the OBB becomes smaller. If not, stop there.
- ▶ The above optimization works quite well in practice.
- ▶ See the code in the text for how to find the minimum-area bounding rectangle.

▶ 13

Sphere-swept Volumes (1)

- ▶ The cylinder is not a good bounding volume.
- ▶ For an arbitrary inner primitive, the Minkowski sum of the inner primitive and a sphere results in a sphere-swept volume (SSV).

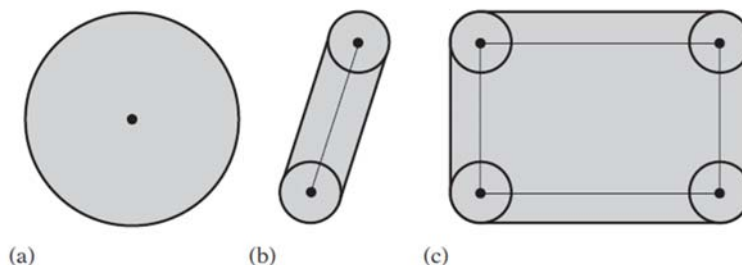


Figure 4.11 (a) A sphere-swept point (SSP). (b) A sphere-swept line (SSL). (c) A sphere-swept rectangle (SSR).

- ▶ All points within a distance r from the inner medial structure belong to the SSV.
- ▶ This greatly simplifies overlap tests.

▶ 14

Sphere-swept Volumes (2)

► Data structure of capsules and lozenges

```
// Region  $R = \{ x \mid (x - [a + (b - a)*t])^2 \leq r \}, 0 \leq t \leq 1$ 
struct Capsule {
    Point a;      // Medial line segment start point
    Point b;      // Medial line segment endpoint

    float r;      // Radius
};

// Region  $R = \{ x \mid (x - [a + u[0]*s + u[1]*t])^2 \leq r \}, 0 \leq s, t \leq 1$ 
struct Lozenge {
    Point a;      // Origin
    Vector u[2];  // The two edges axes of the rectangle
    float r;      // Radius
};
```

► 15

Sphere-swept Volume Intersection

- By construction, all sphere-swept volume tests can be formulated in the same way.
- First, the distance between the inner structures is computed.
- Then this distance is compared against the sum of the radii.
- The only difference between any two types of sphere-swept tests is in the calculation used to compute the distance between the inner structures of the two volumes.

```
int TestSphereCapsule(Sphere s, Capsule capsule)
{
    // Compute (squared) distance between sphere center and capsule line segment
    float dist2 = SqDistPointSegment(capsule.a, capsule.b, s.c);
    // If (squared) distance smaller than (squared) sum of radii, they collide
    float radius = s.r + capsule.r;
    return dist2 <= radius * radius;
}
```

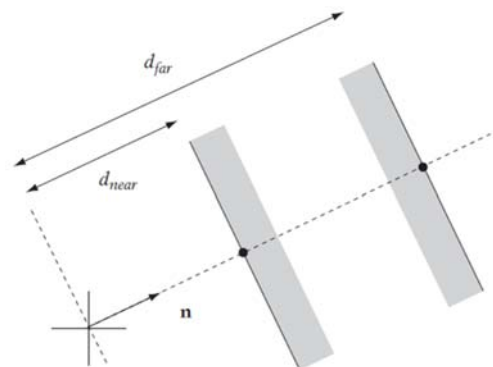
► 16

Computing Sphere-swept Bounding Volumes

- ▶ SSVs can be computed similar to OBBs.
- ▶ See the text for further references to detailed algorithms.

Halfspace Intersection Volumes

- ▶ Convex hulls form the tightest bounding volumes.
- ▶ However, they have some drawbacks; they are expensive and difficult to compute, take large amounts of memory to represent, and potentially are costly to operate upon.
- ▶ By limiting the number of halfspaces used in the intersection volume, several simpler alternative bounding volumes can be formed.
- ▶ They often use slabs.
 - ▶ A slab is the infinite region of space between two parallel planes.
 - ▶ A slab is represented by a unit normal vector and two signed distances from the origin.



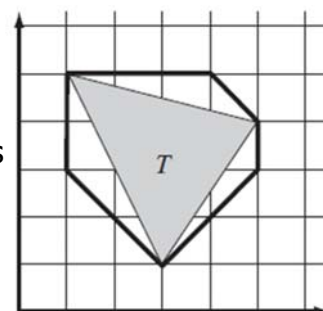
Discrete-orientation Polytopes (k-DOPs) (1)

- ▶ To form a closed 3D volume, at least three slabs are required.
- ▶ By increasing the number of slabs, slab-based bounding volumes can be made to fit the convex hulls of objects arbitrarily tightly.
- ▶ Although slab-based volumes allow for fast ray intersection tests, they cannot easily be used as-is in object-object intersection testing.
- ▶ However, by sharing normals across all objects it is possible to perform fast object-object tests.
- ▶ These are motivations for discrete-orientation polytopes (k-DOPS).

▶ 19

Discrete-orientation Polytopes (k-DOPs) (2)

- ▶ Also called fixed-direction hulls (FDHs)
- ▶ Slab-based volumes where the same normals are shared among all k-DOP bounding volumes
- ▶ The normal components are typically limited to the set $\{-1, 0, 1\}$, and the normals are not normalized.
- ▶ Examples
 - ▶ 6-DOP: Polytopes with faces aligned along the six directions $(\pm 1, 0, 0)$, $(0, \pm 1, 0)$, and $(0, 0, \pm 1)$. That is, an AABB.
 - ▶ 8-DOP: Faces aligned with the eight directions $(\pm 1, \pm 1, \pm 1)$
 - ▶ 12-DOP: Faces aligned with the 12 directions $(\pm 1, \pm 1, 0)$, $(\pm 1, 0, \pm 1)$, and $(0, \pm 1, \pm 1)$.
 - ▶ 14-DOP: Defined using the combined directions of the 6-DOP and 8-DOP. An AABB with the eight corners cut off.
 - ▶ 18-DOP: Defined using the combined directions of the 6-DOP and 12-DOP. An AABB wherein the 12 edges have been cut off.
 - ▶ 20-DOP and 26-DOP are formed in a similar way.



▶ 20

Discrete-orientation Polytopes (k-DOPs) (3)

- ▶ The k-DOP is the tightest slabs that form the body.
- ▶ Compared to OBBs, the overlap test for k-DOPs is much faster (about an order of magnitude), even for large-numbered DOPs, thanks to the fixed direction planes.
- ▶ In terms of storage, the same amount of memory required for an OBB roughly corresponds to a 14-DOP.
- ▶ k-DOPs have a (probable) tighter fit than OBBs, certainly as k grows and the k-DOP resembles the convex hull.
- ▶ For geometry that does not align well with the axes of the k-DOP, OBBs can still be tighter.
- ▶ OBBs will also perform better in situations of close proximity.
- ▶ The largest drawback to k-DOPs is that even if the volumes are rarely colliding the k-DOPs must still be updated, but this operation is expensive.
- ▶ In general, k-DOPs perform best when there are few dynamic objects being tested against many static objects (few k-DOPs have to update) or when the same object is involved in several tests (the cost of updating the k-DOP is amortized over the tests).
- ▶ Not very popular for haptic rendering.
- ▶ For further details, see the text.

▶ 21

Reading

- ▶ Reading material
 - ▶ Ericson, C. Real-Time Collision Detection Morgan Kaufmann Publishers, Ch 4, 2005.
- ▶ Assignments
 - ▶ Write a brief summary on the book chapter.
- ▶ Due date: One week later

▶ 22